ALBUKHARY INTERNATIONAL UNIVERSITY

# CCC2133 : Data Structure and Algorithm Analysis

## Assignment 1 – Individual Assignment

### Linked List Assignment - Java (Student Data Structure with Analysis)


**Student Name: Eylaf Mahdi Adam Saad Elnour**

**Student ID: AIU23102201**

**Group: B**

## Linked List Operations

| Method | Time Complexity | Explanation |
|---|---|---|
| addFirst() | O(1) | Inserts at the head, no traversal needed. Complexity is constant, independent of list size. |
| addLast() | O(1) with tail, O(n) without tail | With a tail pointer (as in my code), insertion is immediate: connect tail.next and update tail. Without tail, we traverse the list to find the last node, which takes O(n). |
| insertAt(index, ...) | O(1) for index 0, O(n) for middle/end | Inserting at head is constant. Inserting at a specific position requires traversing up to index-1 nodes. . |
| deleteFirst() | O(1) | Removes head node, just updates head pointer. |
| deleteLast() | O(n) | Traverses from head to the second-last node to update tail. |
| deleteAt(index) | O(1) for index 0, O(n) for middle/end | Deleting head is constant. Otherwise, traversal is needed. Deleting tail updates tail pointer. |
| search(name) | O(n) | Must check each node sequentially. Best case: first node (O(1)), worst case: last node or not found (O(n)). |
| display() | O(n) | Visits all nodes once to print info. Time grows linearly with list size. |
| getSize() | O(1) | Size is stored in a variable; returning it is constant time. |

### Position Impact:

Operations on the head are always fast (O(1)).

Operations in the middle or end require traversal → O(n).

deleteLast() is particularly expensive for large lists if there's no tail pointer.

### Scalability:

As list size grows, traversal-dependent operations (insert at middle, delete at end, search) grow linearly. Head operations remain efficient.

## Efficiency Analysis with 1000 student nodes.

| Method | Best Case | Worst Case | Big-O |
|---|---|---|---|
| addFirst() | O(1) | O(1) | O(1) |
| addLast() (with tail pointer) | O(1) | O(1) | O(1) |
| insertAt(index) | O(1) | O(n) | O(n) |
| deleteFirst() | O(1) | O(1) | O(1) |
| deleteLast() | O(n) | O(n) | O(n) |
| deleteAt(index) | O(1) | O(n) | O(n) |
| search(name) | O(1) | O(n) | O(n) |
| display() | O(n) | O(n) | O(n) |
| getSize() | O(1) | O(1) | O(1) |
| generateStudentsTo() | O(n) | O(n) | O(n) |

### Implication:

- Linked lists are ideal for frequent additions/removals at the head.
- Random access is slow due to sequential traversal, which is inefficient for large lists when inserting/searching in the middle.

**Comparison with Other Data Structures**

| Operation | Singly Linked List | ArrayList | Explanation |
|---|---|---|---|
| Add at end | O(1) (when tail pointer exists) | O(1) amortized | Both are generally efficient, but for different reasons. A linked list attaches the new node by changing one pointer (tail.next), no traversal needed if a tail exists. An ArrayList writes directly into the next empty slot. |
| Add at start | O(1) | O(n) | Linked list is much faster here. Adding at the front only changes two pointers (newNode.next = head). ArrayList must shift every existing element to the right to make index 0 empty, so the cost grows with list size. |
| Insert at index (middle) | O(n) | O(n) | Both require linear work, but for different reasons. Linked list must traverse nodes one by one to reach the position. ArrayList reaches the index instantly (O(1)), but must shift all elements after the index to the right, which takes O(n). In practice, ArrayList is usually faster due to contiguous memory (cache-friendly). |
| Delete at index (middle) | O(n) | O(n) | Same reason as insert. Linked list must traverse to the node before the one being deleted. ArrayList must shift all elements after the deletion to fill the gap. Both increase in cost as size grows. |

| | | | |
|---|---|---|---|
| Delete at start | O(1) | O(n) | Removing the head of a linked list is instant because only the head pointer changes. ArrayList must shift all remaining elements left by one position, expensive for large lists. |
| Delete at end | O(n) (no previous pointer) | O(1) | Linked list is slow here. Without a previous pointer, you must traverse the whole list to find the node before the tail to update next = null. ArrayList simply reduces its size counter, instant removal. |
| Search (linear) | O(n) | O(n) | Both perform sequential search. But ArrayList is slightly faster in practice because values are stored contiguously, allowing faster CPU caching. Linked list nodes are scattered in memory, causing slower pointer hopping. |
| Display all elements | O(n) | O(n) | Both visit each element exactly once. ArrayList may run slightly faster due to better memory locality but has the same Big-O complexity. |
| getSize() | O(1) | O(1) | Both constant-time if you maintain a size field. |

-In which cases might an ArrayList be preferred over a Singly Linked List, and in which cases might the linked list be a better choice?

| Situation | Better Data Structure | Why |
|---|---|---|
| Adding at start | Linked List | No shifting required |

| | | |
|---|---|---|
| Adding at end (many times) | Linked List (with tail) | Stable O(1), no resizing |
| Removing at start | Linked List | Constant time |
| Removing at end | ArrayList | Linked list must traverse |
| Random access / get(index) | ArrayList | Direct indexing |
| Frequent inserts in middle | ArrayList (slightly) | Better caching |
| Memory efficiency | ArrayList | Compact contiguous storage |
| Very large data with many inserts/removals anywhere | Linked List | Avoids shifting large blocks |

## Edge Case Behavior

- Inserting or deleting from an empty list.

**Both O(1)**

- Searching for a student that does not exist.

**Worst-case O(n) (traverse entire list); best-case O(1) if first node matches.**

- Performing operations on lists with a single node.

**O(1) for deletion,  insertion behaves normally.**

```
 Test Case 2: Edge Cases
Index out of bounds or list is empty.
Index out of bounds or list is empty.
Student 'NoStudent' not found in the list.
```

**Optimization Opportunity:**

If you had to optimize the addLast() operation for larger lists, how would you approach it? Would adding a tail pointer improve efficiency?

**In my project a tail is already used to optamize performance.**

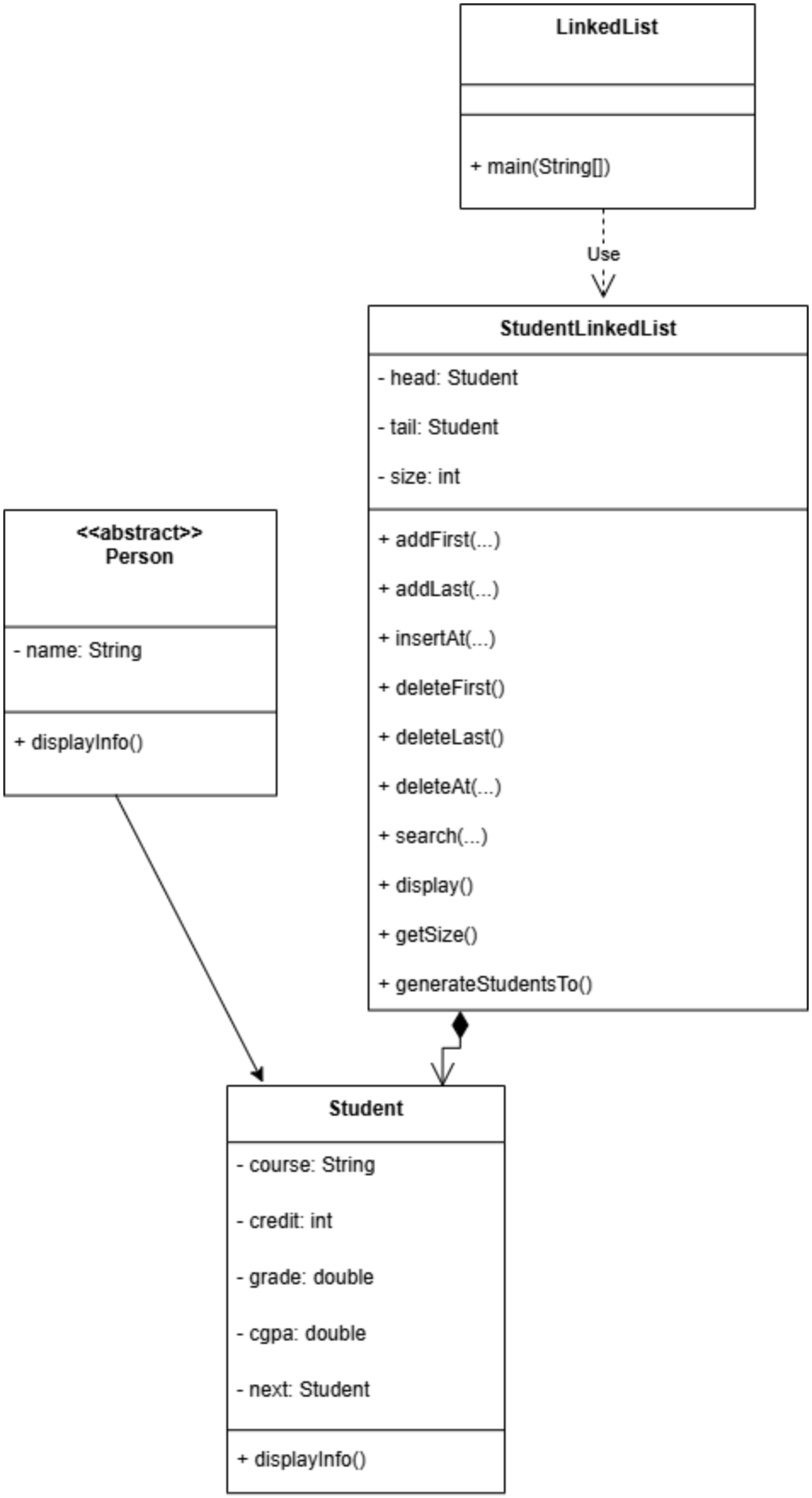
o Insert a student: Bob Brown, CGPA: 3.5 at index 1.

o Delete first student.

```
After adding students:
Bob Brown            with CGPA: 3.50
Ali Bin Ahmad        with CGPA: 3.42
InsertedName         with CGPA: 3.00
Siti Nur             with CGPA: 3.80
Kumar                with CGPA: 2.95
Fatimah Hassan       with CGPA: 3.55
John Lee             with CGPA: 3.10
Amina Yusuf          with CGPA: 3.90
Mohamed Saleh        with CGPA: 2.70
Nurul Iman           with CGPA: 3.35
Hassan Ali           with CGPA: 3.00
Samantha Tan         with CGPA: 3.60
Eyman Adam           with CGPA: 3.80
Total students: 13

After deleting students:
Ali Bin Ahmad        with CGPA: 3.42
Siti Nur             with CGPA: 3.80
Kumar                with CGPA: 2.95
```

**UML Diagram:**


LinkedList
+ main(String[])
Use
StudentLinkedList
- head: Student
- tail: Student
- size: int
+ addFirst(...)
+ addLast(...)
+ insertAt(...)
+ deleteFirst()
+ deleteLast()
+ deleteAt(...)
+ search(...)
+ display()
+ getSize()
+ generateStudentsTo()
<<abstract>>
Person
- name: String
+ displayInfo()
Student
- course: String
- credit: int
- grade: double
- cgpa: double
- next: Student
+ displayInfo()